

```
In [9]: import pandas as pd
import numpy as np
```

```
In [11]: s = pd.Series([1, 3, 5, np.nan, 6, 8])
[11]: s
```

```
Out[11]: 0    1.0
         1    3.0
         2    5.0
         3    NaN
         4    6.0
         5    8.0
         dtype: float64
```

```
In [13]: dates = pd.date_range("20130101", periods=6)
[13]: dates
```

```
Out[13]: DatetimeIndex(['2013-01-01', '2013-01-02', '2013-01-03',
                        '2013-01-04',
                        '2013-01-05', '2013-01-06'],
                        dtype='datetime64[ns]', freq='D')
```

```
In [15]: df = pd.DataFrame(np.random.randn(6, 4), index=dates,
[15]: columns=list("ABCD"))
df
```

```
Out[15]:
```

	A	B	C	D
2013-01-01	-1.899677	0.962794	1.295796	1.356394
2013-01-02	-0.098544	0.515543	-1.903747	-1.034082
2013-01-03	-1.006372	-0.803264	0.811543	1.640434
2013-01-04	2.952784	0.617633	-1.217763	-0.952275
2013-01-05	0.112889	0.952576	-0.232449	-1.247942
2013-01-06	-1.081564	0.724197	0.668549	0.573190

```
In [ ]: df2 = pd.DataFrame(
    {
        "A": [1.0, 2.0, 3.0, 4.0],
        "B": pd.Timestamp("20130102"),
        "C": pd.Series(1, index=list(range(4)),
dtype="float32"),
        "D": np.array([3] * 4, dtype="int32"),
        "E": pd.Categorical(["test", "train", "test",
"train"]),
        "F": "foo",
    }
)

df2
```

	A	B	C	D	E	F
0	1.0	2013-01-02	1.0	3	test	foo
1	2.0	2013-01-02	1.0	3	train	foo
2	3.0	2013-01-02	1.0	3	test	foo
3	4.0	2013-01-02	1.0	3	train	foo

```
In
[22]: df2 = pd.DataFrame(
    {
        "A": 1.0,
        "B": pd.Timestamp("20130102"),
        "C": pd.Series(1, index=list(range(4)),
dtype="float32"),
        "D": np.array([3] * 4, dtype="int32"),
        "E": pd.Categorical(["test", "train", "test",
"train"]),
        "F": "foo",
    }
)

df2
```

Out[22]:

	A	B	C	D	E	F
0	1.0	2013-01-02	1.0	3	test	foo
1	1.0	2013-01-02	1.0	3	train	foo
2	1.0	2013-01-02	1.0	3	test	foo
3	1.0	2013-01-02	1.0	3	train	foo

In `df.head()`

[23]:

Out[23]:

	A	B	C	D
2013-01-01	-1.899677	0.962794	1.295796	1.356394
2013-01-02	-0.098544	0.515543	-1.903747	-1.034082
2013-01-03	-1.006372	-0.803264	0.811543	1.640434
2013-01-04	2.952784	0.617633	-1.217763	-0.952275
2013-01-05	0.112889	0.952576	-0.232449	-1.247942

In `df.tail(3)`

[24]:

Out[24]:

	A	B	C	D
2013-01-04	2.952784	0.617633	-1.217763	-0.952275
2013-01-05	0.112889	0.952576	-0.232449	-1.247942
2013-01-06	-1.081564	0.724197	0.668549	0.573190

In `df.index`

[25]:

Out[25]: DatetimeIndex(['2013-01-01', '2013-01-02', '2013-01-03',
'2013-01-04',
'2013-01-05', '2013-01-06'],
dtype='datetime64[ns]', freq='D')

In `df.columns`

[26]:

Out[26]: Index(['A', 'B', 'C', 'D'], dtype='object')

In `df.to_numpy()`

[29]:

```
Out[29]: array([[ -1.89967683,  0.96279418,  1.29579602,  1.3563936
                ],
                [ -0.09854421,  0.51554326, -1.90374719,
                -1.03408151],
                [ -1.00637243, -0.80326358,  0.81154335,
                1.64043406],
                [  2.95278353,  0.6176333 , -1.2177626 ,
                -0.95227459],
                [  0.11288888,  0.95257589, -0.23244865,
                -1.24794154],
                [ -1.08156437,  0.72419672,  0.66854851,  0.5731903
                ]])
```

In `df2.dtypes`

[30]:

```
Out[30]: A      float64
        B  datetime64[s]
        C    float32
        D      int32
        E    category
        F      object
        dtype: object
```

In `df2.to_numpy()`

[31]:

```
Out[31]: array([[1.0, Timestamp('2013-01-02 00:00:00'), 1.0, 3,
                'test', 'foo'],
                [1.0, Timestamp('2013-01-02 00:00:00'), 1.0, 3,
                'train', 'foo'],
                [1.0, Timestamp('2013-01-02 00:00:00'), 1.0, 3,
                'test', 'foo'],
                [1.0, Timestamp('2013-01-02 00:00:00'), 1.0, 3,
                'train', 'foo']],
                dtype=object)
```

```
In df.describe()
```

```
[32]:
```

```
Out[32]:
```

	A	B	C	D
count	6.000000	6.000000	6.000000	6.000000
mean	-0.170081	0.494913	-0.096345	0.055953
std	1.694648	0.660558	1.256199	1.294148
min	-1.899677	-0.803264	-1.903747	-1.247942
25%	-1.062766	0.541066	-0.971434	-1.013630
50%	-0.552458	0.670915	0.218050	-0.189542
75%	0.060031	0.895481	0.775795	1.160593
max	2.952784	0.962794	1.295796	1.640434

```
In df.T
```

```
[33]:
```

```
Out[33]:
```

	2013-01-01	2013-01-02	2013-01-03	2013-01-04	2013-01-05
A	-1.899677	-0.098544	-1.006372	2.952784	0.112889
B	0.962794	0.515543	-0.803264	0.617633	0.952576
C	1.295796	-1.903747	0.811543	-1.217763	-0.232449
D	1.356394	-1.034082	1.640434	-0.952275	-1.247942

```
In df.sort_index(axis=1, ascending=False)
```

```
[34]:
```

```
Out[34]:
```

	D	C	B	A
2013-01-01	1.356394	1.295796	0.962794	-1.899677
2013-01-02	-1.034082	-1.903747	0.515543	-0.098544
2013-01-03	1.640434	0.811543	-0.803264	-1.006372
2013-01-04	-0.952275	-1.217763	0.617633	2.952784
2013-01-05	-1.247942	-0.232449	0.952576	0.112889
2013-01-06	0.573190	0.668549	0.724197	-1.081564

```
In [37]: df.sort_index(axis=1, ascending = True)
```

Out[37]:

	A	B	C	D
2013-01-01	-1.899677	0.962794	1.295796	1.356394
2013-01-02	-0.098544	0.515543	-1.903747	-1.034082
2013-01-03	-1.006372	-0.803264	0.811543	1.640434
2013-01-04	2.952784	0.617633	-1.217763	-0.952275
2013-01-05	0.112889	0.952576	-0.232449	-1.247942
2013-01-06	-1.081564	0.724197	0.668549	0.573190

```
In [38]: df.sort_values(by = "B")
```

Out[38]:

	A	B	C	D
2013-01-03	-1.006372	-0.803264	0.811543	1.640434
2013-01-02	-0.098544	0.515543	-1.903747	-1.034082
2013-01-04	2.952784	0.617633	-1.217763	-0.952275
2013-01-06	-1.081564	0.724197	0.668549	0.573190
2013-01-05	0.112889	0.952576	-0.232449	-1.247942
2013-01-01	-1.899677	0.962794	1.295796	1.356394

```
In [40]: df["A"]
```

Out[40]:

```
2013-01-01    -1.899677
2013-01-02    -0.098544
2013-01-03    -1.006372
2013-01-04     2.952784
2013-01-05     0.112889
2013-01-06    -1.081564
Freq: D, Name: A, dtype: float64
```

In `df.A`

[41]:

```
Out[41]: 2013-01-01    -1.899677
          2013-01-02    -0.098544
          2013-01-03    -1.006372
          2013-01-04     2.952784
          2013-01-05     0.112889
          2013-01-06    -1.081564
          Freq: D, Name: A, dtype: float64
```

In `df[['A', 'B']]`

[43]:

```
Out[43]:
```

	A	B
2013-01-01	-1.899677	0.962794
2013-01-02	-0.098544	0.515543
2013-01-03	-1.006372	-0.803264
2013-01-04	2.952784	0.617633
2013-01-05	0.112889	0.952576
2013-01-06	-1.081564	0.724197

In `df[0:3]`

[46]:

```
Out[46]:
```

	A	B	C	D
2013-01-01	-1.899677	0.962794	1.295796	1.356394
2013-01-02	-0.098544	0.515543	-1.903747	-1.034082
2013-01-03	-1.006372	-0.803264	0.811543	1.640434

In `df["20130102":"20130104"]`

[47]:

```
Out[47]:
```

	A	B	C	D
2013-01-02	-0.098544	0.515543	-1.903747	-1.034082
2013-01-03	-1.006372	-0.803264	0.811543	1.640434
2013-01-04	2.952784	0.617633	-1.217763	-0.952275

```
In [49]: dfl = pd.DataFrame(np.random.randn(5, 4), columns=list('ABCD'),  
index=pd.date_range('20130101', periods=5))
```

```
In [50]: dfl
```

```
Out[50]:
```

	A	B	C	D
2013-01-01	-0.149486	-1.715496	0.188584	0.202803
2013-01-02	0.491150	0.354022	-2.032739	0.398700
2013-01-03	-1.239317	1.325217	0.933295	-0.881066
2013-01-04	1.045198	1.193305	-0.314060	-0.339409
2013-01-05	0.589993	-1.162830	-1.271066	2.338282

```
In [51]: dfl.loc['20130102':'20130104']
```

```
Out[51]:
```

	A	B	C	D
2013-01-02	0.491150	0.354022	-2.032739	0.398700
2013-01-03	-1.239317	1.325217	0.933295	-0.881066
2013-01-04	1.045198	1.193305	-0.314060	-0.339409

```
In [54]: s1 = pd.Series(np.random.randn(6), index=list('abcdef'))  
s1
```

```
Out[54]: a    0.421753  
b    1.000201  
c    0.209521  
d   -1.019046  
e    0.019487  
f   -0.593953  
dtype: float64
```



```
In      s1.loc['c':]
[55]:
```

```
Out[55]: c      0.209521
         d     -1.019046
         e      0.019487
         f     -0.593953
         dtype: float64
```

```
In      df1 = pd.DataFrame(np.random.randn(6, 4),
[57]:                                index=list('abcdef'),
                                   columns=list('ABCD'))
```

```
In      df1.loc['a'] > 0
[58]:
```

```
Out[58]: A      True
         B      True
         C     False
         D     False
         Name: a, dtype: bool
```

```
In      mask = pd.array([True, False, True, False, pd.NA, False],
[61]:                      dtype="boolean")
```

```
In      mask
[62]:
```

```
Out[62]: <BooleanArray>
         [True, False, True, False, <NA>, False]
         Length: 6, dtype: boolean
```

```
In      df1[mask]
[64]:
```

```
Out[64]:
```

	A	B	C	D
a	1.284777	0.890076	-0.700467	-0.629934
c	-0.426258	0.497475	-1.093577	-0.993922

```
In      arrays = [
[65]:          ["bar", "bar", "baz", "baz", "foo", "foo", "qux", "qux"],
          ["one", "two", "one", "two", "one", "two", "one", "two"],
          ]
```

```
In      tuples = list(zip(*arrays))
[68]:   tuples
```

```
Out[68]: [('bar', 'one'),
          ('bar', 'two'),
          ('baz', 'one'),
          ('baz', 'two'),
          ('foo', 'one'),
          ('foo', 'two'),
          ('qux', 'one'),
          ('qux', 'two')]
```

```
In [ ]: index = pd.MultiIndex.from_tuples(tuples, name = ['first',
'second'])
index
```

```
MultiIndex([('bar', 'one'),
            ('bar', 'two'),
            ('baz', 'one'),
            ('baz', 'two'),
            ('foo', 'one'),
            ('foo', 'two'),
            ('qux', 'one'),
            ('qux', 'two')],
           names=['first', 'second'])
```

```
In      s = pd.Series(np.random.randn(8), index = index)
[110]:   s
```

```
Out[110]:first  second
bar      one    -0.188457
          two    -1.187526
baz      one    -0.977592
          two    -0.214904
foo      one    -1.241294
          two    -1.368656
qux      one    -1.439867
          two     1.544199
dtype: float64
```

In []: #When you want every pairing of the elements in two iterables, it can be easier to use the `MultiIndex.from_product()` method:

```
iterables = [['bar', 'baz', 'foo', 'qux'], ['one', 'two']]

pd.MultiIndex.from_product(iterables, names = ['first',
'second'])
```

```
MultiIndex([('bar', 'one'),
            ('bar', 'two'),
            ('baz', 'one'),
            ('baz', 'two'),
            ('foo', 'one'),
            ('foo', 'two'),
            ('qux', 'one'),
            ('qux', 'two')],
           names=['first', 'second'])
```

In []: #You can also construct a `MultiIndex` from a `DataFrame` directly, using the method `MultiIndex.from_frame()`. This is a complementary method to `MultiIndex.to_frame()`.

```
df = pd.DataFrame(
    [['bar', 'one'], ['bar', 'two'], ['foo', 'one'], ['foo',
'two']],
    columns = ['first', 'second']
)

pd.MultiIndex.from_frame(df)
```

```
MultiIndex([('bar', 'one'),
            ('bar', 'two'),
            ('foo', 'one'),
            ('foo', 'two')],
           names=['first', 'second'])
```

In []: #As a convenience, you can pass a list of arrays directly into `Series` or `DataFrame` to construct a `MultiIndex` automatically:

```
In
[102]: arrays = [np.array(['bar', 'bar', 'baz', 'baz', 'foo', 'foo',
'qux', 'qux']),
                np.array(['one', 'two', 'one', 'two', 'one', 'two',
'one', 'one'])]
```

```
In [103]: s = pd.Series(np.random.randn(8), index = arrays)
s
```

```
Out[103]: bar one -0.181343
          two  0.063864
          baz one  0.245476
          two  0.525193
          foo one -0.397125
          two -1.512212
          qux one -0.979524
          one -0.891076
          dtype: float64
```

```
In [104]: df = pd.DataFrame(np.random.randn(8, 4), index = arrays)
df
```

```
Out[104]:
```

		0	1	2	3
bar	one	1.204708	0.951528	-0.587728	0.616754
	two	1.189993	0.132244	0.500720	0.629701
baz	one	-2.493601	0.144110	-0.342211	-0.433439
	two	1.444636	1.339213	0.538231	-1.151400
foo	one	0.093020	-0.070525	0.198078	0.373220
	two	0.465991	1.647930	1.026402	1.076880
qux	one	-0.851770	-0.014532	-0.974419	1.171885
	one	0.550368	0.599783	0.814637	0.444525

```
In [ ]: df.index.names
#All of the MultiIndex constructors accept a names argument
which stores string names for the levels themselves. If no
names are provided, None will be assigned
```

```
FrozenList([None, None])
```

```
In [ ]: #This index can back any axis of a pandas object, and the
        number of levels of the index is up to you:

        df = pd.DataFrame(np.random.randn(6, 6), index = index[:6],
                           columns = index[:6])
        df
```

	first	bar		baz	
	second	one	two	one	two
first	second				
bar	one	-0.732588	0.369999	-0.394761	1.471708
	two	-0.255188	-0.453368	-0.083779	-0.103571
baz	one	2.436550	0.401351	0.115168	0.570896
	two	0.321968	0.139914	1.327352	-0.653767
foo	one	-0.350050	-1.815670	0.508437	0.522383
	two	0.311271	-1.612044	0.385821	0.011667

```
In
[107]: df = pd.DataFrame(np.random.randn(3, 8), index = (['A', 'B',
                                                         'C']), columns = index)
        df
```

```
Out[107]:
```

	first	bar		baz		foo
	second	one	two	one	two	
A		0.652610	-0.024621	0.625762	-0.257216	-0.414
B		-1.277653	-1.362059	-2.149282	-0.244885	-0.719
C		1.099913	-0.989699	0.808137	-2.095750	-0.900

```
In [ ]: #We've "sparsified" the higher levels of the indexes to make
        the console output
        #a bit easier on the eyes. Note that how the index is displayed
        can be controlled
        #using the multi_sparse option in pandas.set_options():

        with pd.option_context("display.multi_sparse", False):
            df
```

```
In [112]: #It's worth keeping in mind that
#there's nothing preventing you from using tuples as atomic
labels on an axis:

pd.Series(np.random.randn(8), index = tuples)
```

```
Out[112]: (bar, one)    -0.147092
          (bar, two)    1.433907
          (baz, one)    -0.470914
          (baz, two)    0.440032
          (foo, one)    -1.189244
          (foo, two)    1.360071
          (qux, one)    -0.027156
          (qux, two)    0.948596
          dtype: float64
```

```
In [ ]: #The method get_level_values() will return a vector of the
labels for each location at a particular level:
index.get_level_values('second')
```

```
Index(['one', 'two', 'one', 'two', 'one', 'two', 'one',
      'two'], dtype='object', name='second')
```

```
In [115]: index.get_level_values(0)
```

```
Out[115]: Index(['bar', 'bar', 'baz', 'baz', 'foo', 'foo', 'qux',
                'qux'], dtype='object', name='first')
```

```
In [117]: #The MultiIndex keeps all the defined levels of an index,
# even if they are not actually used. When slicing an index,
you may notice this. For example:
```

```
df.columns.levels

df[['foo', 'qux']].columns.levels
```

```
Out[117]: FrozenList([['bar', 'baz', 'foo', 'qux'], ['one', 'two']])
```

```
In [ ]: #This is done to avoid a recomputation of the levels in order
        # to make slicing highly performant.
        # If you want to see only the used levels, you can use the
        # get_level_values() method.
        df[['foo', 'qux']].columns.to_numpy()
```

```
array([('foo', 'one'), ('foo', 'two'), ('qux', 'one'),
      ('qux', 'two')],
      dtype=object)
```

```
In [119]: df[['foo', 'qux']].columns.get_level_values(0)
```

```
Out[119]: Index(['foo', 'foo', 'qux', 'qux'], dtype='object',
                name='first')
```

```
In [121]: #To reconstruct the MultiIndex with only the used levels,
        # the remove_unused_levels() method may be used.

        new_mi = df[['foo', 'qux']].columns.remove_unused_levels()

        new_mi.levels
```

```
Out[121]: FrozenList(['foo', 'qux'], ['one', 'two'])
```

```
In [124]: #Operations between differently-indexed objects having
        # MultiIndex on the axes will work as you expect;
        # data alignment will work the same as an Index of tuples:

        s + s[:-2]
```

```
Out[124]: first  second
          bar      one      -0.376914
          two      -2.375052
          baz      one      -1.955183
          two      -0.429808
          foo      one      -2.482587
          two      -2.737312
          qux      one         NaN
          two         NaN
dtype: float64
```

In [123]: `s + s[:,2]`

```
Out[123]:first  second
      bar    one    -0.376914
          two      NaN
      baz    one   -1.955183
          two      NaN
      foo    one   -2.482587
          two      NaN
      qux    one   -2.879734
          two      NaN
      dtype: float64
```

In [126]: `#The reindex() method of Series/DataFrames can be called with another MultiIndex, or even a list or array of tuples:
s.reindex(index[:3])`

```
Out[126]:first  second
      bar    one    -0.188457
          two    -1.187526
      baz    one    -0.977592
      dtype: float64
```

In [128]: `s.reindex([('foo', 'two'), ('bar', 'one'), ('qux', 'one'), ('baz', 'one')])`

```
Out[128]:first  second
      foo    two    -1.368656
      bar    one    -0.188457
      qux    one    -1.439867
      baz    one    -0.977592
      dtype: float64
```

In [129]: `# Syntactically integrating MultiIndex in advanced indexing with .loc is a bit challenging, but we've made every effort to do so. In general, MultiIndex keys take the form of tuples.
For example, the following works as you would expect:

df = df.T`

In
[130]:

df

Out[130]:

		A	B	C
	first	second		
bar	one	0.652610	-1.277653	1.099913
	two	-0.024621	-1.362059	-0.989699
baz	one	0.625762	-2.149282	0.808137
	two	-0.257216	-0.244885	-2.095750
foo	one	-0.414284	-0.719126	-0.900155
	two	0.725423	-0.518687	-0.659393
qux	one	-0.067630	0.780362	-0.609348
	two	0.146955	-3.093812	2.011608

In
[133]:

```
df1 = pd.DataFrame(
    {
        'A' : ['A0', 'A1', 'A2', 'A3'],
        'B' : ['B0', 'B1', 'B2', 'B3'],
        'C' : ['C0', 'B1', 'C2', 'C3'],
        'D' : ['D0', 'D1', 'D2', 'D3']
    }, index = [0, 1, 2, 3]
)

df2 = pd.DataFrame(
    {
        "A": ["A4", "A5", "A6", "A7"],
        "B": ["B4", "B5", "B6", "B7"],
        "C": ["C4", "C5", "C6", "C7"],
        "D": ["D4", "D5", "D6", "D7"],
    },
    index=[4, 5, 6, 7],
)

df3 = pd.DataFrame(
    {
        "A": ["A8", "A9", "A10", "A11"],
        "B": ["B8", "B9", "B10", "B11"],
        "C": ["C8", "C9", "C10", "C11"],
        "D": ["D8", "D9", "D10", "D11"],
    },
    index=[8, 9, 10, 11],
)
```

```
frames = [df1, df2, df3]

result = pd.concat(frames)

result
```

Out[133]:

	A	B	C	D
0	A0	B0	C0	D0
1	A1	B1	B1	D1
2	A2	B2	C2	D2
3	A3	B3	C3	D3
4	A4	B4	C4	D4
5	A5	B5	C5	D5
6	A6	B6	C6	D6
7	A7	B7	C7	D7
8	A8	B8	C8	D8
9	A9	B9	C9	D9
10	A10	B10	C10	D10
11	A11	B11	C11	D11

```
In [135]: df4 = pd.DataFrame(
    {
        "B": ["B2", "B3", "B6", "B7"],
        "D": ["D2", "D3", "D6", "D7"],
        "F": ["F2", "F3", "F6", "F7"],
    },
    index=[2, 3, 6, 7],
)

result = pd.concat([df1, df4], axis=1)

result
```

```
Out[135]:
```

	A	B	C	D	B	D	F
0	A0	B0	C0	D0	NaN	NaN	NaN
1	A1	B1	B1	D1	NaN	NaN	NaN
2	A2	B2	C2	D2	B2	D2	F2
3	A3	B3	C3	D3	B3	D3	F3
6	NaN	NaN	NaN	NaN	B6	D6	F6
7	NaN	NaN	NaN	NaN	B7	D7	F7

```
In [136]: #join='inner' takes the intersection of the axis values.

result = pd.concat([df1, df4], axis = 1, join = 'inner')
result
```

```
Out[136]:
```

	A	B	C	D	B	D	F
2	A2	B2	C2	D2	B2	D2	F2
3	A3	B3	C3	D3	B3	D3	F3

```
In [138]: #To perform an effective "left" join using the exact index
# from the original DataFrame, result can be reindexed.

result = pd.concat([df1, df4], axis = 1).reindex(df1.index)

result
```

Out[138]:

	A	B	C	D	B	D	F
0	A0	B0	C0	D0	NaN	NaN	NaN
1	A1	B1	B1	D1	NaN	NaN	NaN
2	A2	B2	C2	D2	B2	D2	F2
3	A3	B3	C3	D3	B3	D3	F3

In []:

Exported with [runcell](https://www.runcell.dev) — convert notebooks to HTML or PDF anytime at [runcell.dev](https://www.runcell.dev).